

Definition of an ERROR.

An Error is a Numerical computation is the difference b/w the actual value of a quantity and its computed (Approximated value). If x^* represent the computed value of a quantity, the actual value for which is x the difference $e = |x - x^*|$ is called the error of approximation.

Sources of ERRORS.

1) Gross ERROR

2) Rounding "

3) Truncation =

(Drop) - Chopping Error

0.001 of distance screw

1) Gross ERRORS

^{Int}

->

Basic

Basic assumption

0.7625

The gross Error are either caused by human mistakes or by the computer.

1) Misreading the figures

2) Use of inaccurate mathematical formula (Algorithm)

3) Use of inaccurate data.

2) Rounding ERRORS.

The Error introduced by rounding off numbers to a limited number of decimal places is called the rounding error.

exp $x = 3.141592654 = x$
 $= 3.142 = x^*$

$$e = |x - x^*| = |3.141592654 - 3.142| =$$

$$e = 0.000407346$$

Rounding Error Rules:

A) if the first discarded digit is less than 5, the previous digit is unchanged; For exp.

$$56.44 \rightarrow 56.4$$

B) if the first discarded digit is greater than 5, the previous digit is increased by 1, For exp.

$$56.46 \rightarrow 56.5$$

C) If the discarded digit is exactly 5, the previous digit is unchanged if it is even and is increased by 1 if it is odd, For exp.

$$(i) \quad 56.45 \rightarrow 56.4 \quad \text{Even}$$

$$(ii) \quad 56.75 \rightarrow 56.8 \quad \text{Odd}$$

Truncation Errors

The error is due to truncating the series is called truncation error

$$e^x = 1 + x + \frac{x^2}{2} + \frac{x^3}{3} + \dots$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} + \dots$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \dots + (-1)^n \frac{x^{2n}}{(2n)!}$$

Types of Error

There are three types of Errors

- 1) Absolute Error (A.E) $\rightarrow$
- 2) Relative = (R.E)
- 3) Percentage = (P.E)

i. Absolute Error.

The difference b/w exact value and its approximate value is called absolute error.

$$\Delta x = A.E = | \text{Exact} - \text{Approximate} | = | x - x^* |$$

$$\begin{aligned} \text{eg } T_2 &= 1.4142 = x \\ &= 1.414 = x^* \end{aligned}$$

$$\Delta x = A.E = | 1.4142 - 1.414 | = 0.0002$$

$$\Delta x = 0.0002$$

2) Relative Error (R.E)

is the ratio b/w absolute error and Exact Value and it is denoted by 'R.E'

$$R.E = \frac{\Delta x}{x}$$

Percentage Error (P.E).

if the relative error expressed in term of % then it will be a % error.

$$P.E = \frac{\Delta x}{x} \times 100\%$$

Chapter 1 Error

What is an Error in Numerical Analysis?

In **numerical analysis**, an **error** is the difference between the **true/exact value** and the **computed/approximated value** of a quantity. It reflects the **inaccuracy** introduced by numerical methods due to limitations like finite precision, approximations, and computational procedures.

$$\text{Error} = \text{True Value} - \text{Computed Value}$$

Applications in Computer Science:

Errors and their analysis are crucial in various **computer science** areas:

- Scientific Computing**
Used in simulations (e.g., weather forecasting, physics modeling) where small errors can accumulate and affect outcomes.
- Computer Graphics**
Approximations in rendering and transformations often involve round-off and truncation errors.
- Machine Learning and AI**
Loss functions are based on error measurements (e.g., Mean Squared Error) to optimize models.
- Cryptography and Numerical Algorithms**
Precise calculations are critical, and error analysis helps ensure stability and accuracy.
- Embedded Systems and Real-Time Systems**
Sensor inputs and floating-point arithmetic introduce errors; systems must be designed to tolerate or correct them.
- Software Engineering**
Numerical methods are used in algorithms where approximations (e.g., root finding, matrix operations) are required.

Why It Is Used:

- **To evaluate accuracy** of numerical methods and algorithms.
- **To choose suitable algorithms** that minimize or control error.
- **To ensure reliability** of computational results in scientific, financial, and engineering applications.
- **To detect instability** or divergence in iterative processes.
- **To optimize performance** by balancing between speed and acceptable precision.

Chapter 4

Why Optimization is Used in Numerical Analysis

Optimization in numerical analysis refers to the process of **finding the best solution** (maximum or minimum) for a problem under given constraints. It's used because many real-world problems require determining the most efficient, cost-effective, or accurate solution from a set of possibilities.

Why It Is Used:

- Resource Efficiency:** To minimize time, cost, or energy consumption.
- Accuracy Improvement:** To minimize error in numerical models.
- Decision Making:** To find the best course of action among alternatives.
- Problem Solving:** Many scientific and engineering problems are naturally optimization problems (e.g., minimizing stress on a bridge).

Applications in Computer Science:

Area	Application of Optimization
Machine Learning & AI	Training models by minimizing loss functions (e.g., gradient descent)
Operating Systems	Process scheduling, memory management (e.g., minimizing turnaround time)
Computer Networks	Optimizing routing paths to minimize latency or maximize throughput
Software Engineering	Code optimization for performance and memory usage
Computer Vision	Image enhancement, object detection by optimizing feature matching
Game Development	Pathfinding (e.g., A* algorithm), AI decisions using utility optimization
Database Systems	Query optimization for faster data retrieval
Cybersecurity	Minimizing risk or maximizing detection accuracy in intrusion systems

1st-Order Method: Gradient Descent

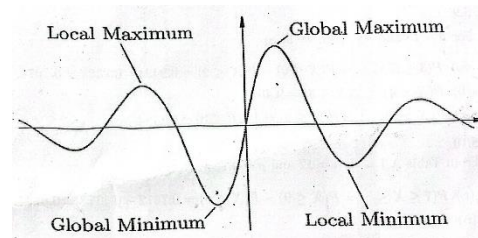
- **Uses:** First derivative (gradient) of the function.
- **Idea:** Moves in the direction of steepest descent (negative gradient).
- **Computation Cost:** Low (only requires gradient).
- **Speed:** Slower convergence, especially near the minimum.
- **Memory Usage:** Efficient; suitable for large-scale problems.
- **Example:** $x_{\text{new}} = x_{\text{old}} - \alpha \nabla f(x)$

2nd-Order Method: Newton's Method (uses Hessian Matrix)

- **Uses:** First and second derivatives (gradient + Hessian matrix).
- **Idea:** Uses curvature information to find a better direction and step size.
- **Computation Cost:** High (Hessian is expensive to compute and invert).
- **Speed:** Faster convergence, especially near the minimum.
- **Memory Usage:** High; not ideal for high-dimensional data.
- **Example:**

$$x_{\text{new}} = x_{\text{old}} - H^{-1} \nabla f(x)$$

where H is the Hessian matrix (matrix of second partial derivatives).



1. Minima (Minimum):

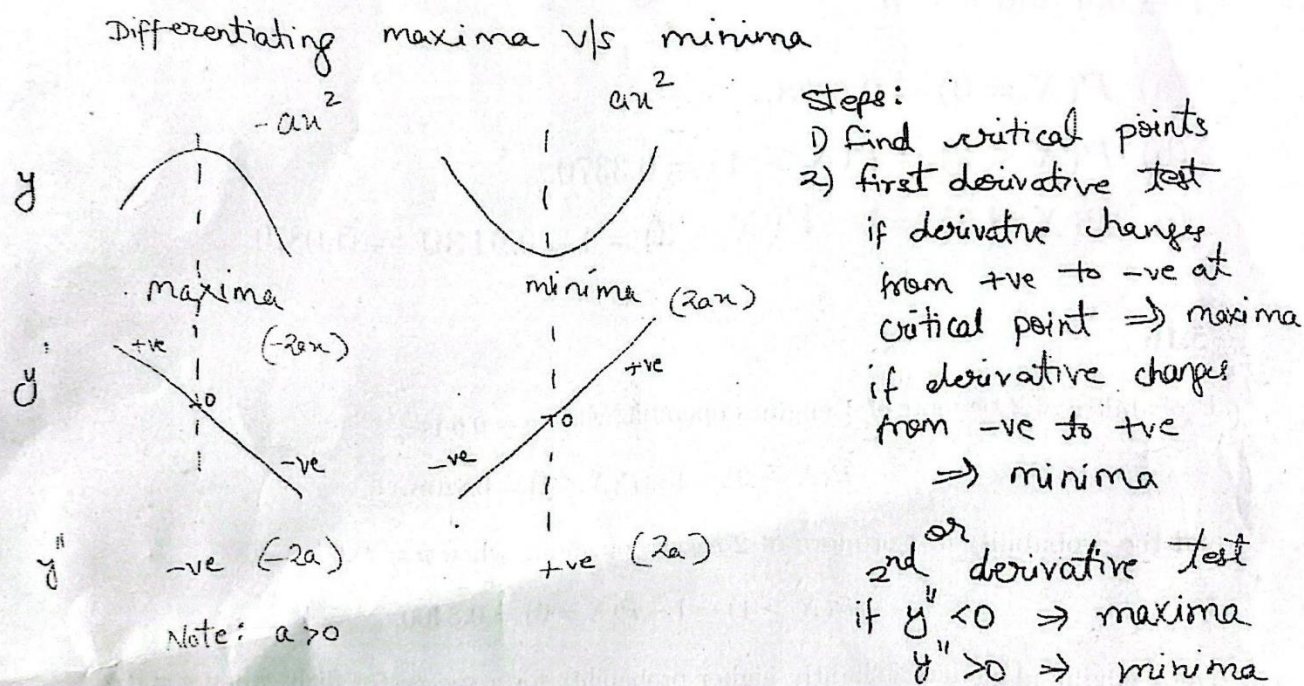
A **minimum** of a function is a point where the function value is **less than or equal to** the values at nearby points.

- **Local Minimum:** A point where the function has the lowest value in a small region.
- **Global Minimum:** A point where the function has the lowest value over its entire domain.

2. Maxima (Maximum):

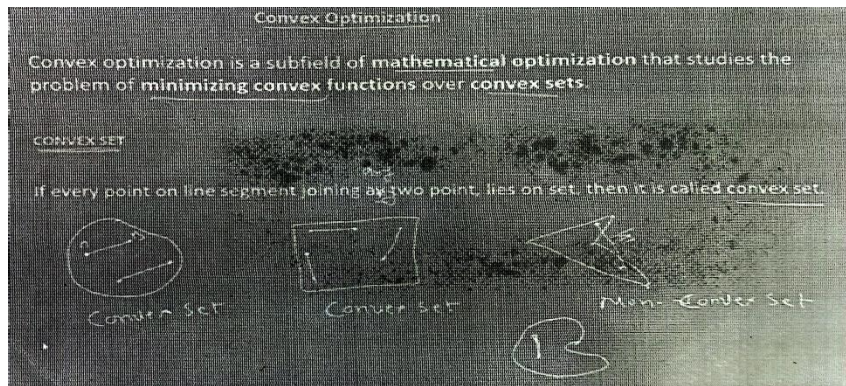
A **maximum** of a function is a point where the function value is **greater than or equal to** the values at nearby points.

- **Local Maximum:** A point where the function has the highest value in a small region.
- **Global Maximum:** A point where the function has the highest value over its entire domain.



Tip: If there are multiple minima, calculate y values and check which is lowest. That point is global minimum and rest are local minima. IIIrd test for maximum.

Concept	Local Minimum	Global Minimum	Local Maximum	Global Maximum
Definition	Smallest value within a neighborhood.	Smallest value over the entire domain.	Largest value within a neighborhood.	Largest value over the entire domain.
Scope	Local region around a point.	Entire domain of the function.	Local region around a point.	Entire domain of the function.
Graph	Appears as a "valley" or dip.	Appears as the lowest point of the graph.	Appears as a "peak" or hill.	Appears as the highest point of the graph.
Example	$f(x) = x^2 + 2$ at $x = 0$	$f(x) = x^2$ at $x = 0$	$f(x) = -x^2 + 3$ at $x = 0$	$f(x) = -x^2$ at $x = 0$



convex optimization problems have a single global minimum and are relatively easier

- **Portfolio Optimization:** linear relationship between the expected returns and the risk (covariance) matrix.
- **Least Squares Regression:** In linear regression, the goal is to find the parameters that minimize the sum of squared differences between the observed and predicted values.
- **Support Vector Machines (SVM):** In binary classification, SVM aims to find the hyperplane that maximizes the margin between the two classes while minimizing classification errors.

non-convex optimization problems may have multiple local minima, making them more challenging to solve,

- **Neural Network Training:** Training deep neural networks involves optimizing a non-convex loss function with respect to the model parameters (weights and biases).
- **Image Registration:** In medical imaging, image registration aims to align different imaging modalities or time-series images. The optimization problem involves finding the transformation parameters that minimize the discrepancy between the images.

Chapter 5 Interpolation

In numerical analysis, **interpolation** refers to the process of estimating the values of a function at points between known values. Given a set of data points, interpolation aims to find a function that "fits" the data and can be used to estimate values at intermediate points.

In simpler terms, interpolation allows us to construct new data points within the range of a discrete set of known data points.

Common types of interpolation methods include:

1. **Linear interpolation:** Assumes the value between two points lies along a straight line.
2. **Polynomial interpolation:** Uses polynomials to fit the data points.
3. **Spline interpolation:** Uses piecewise polynomials (often cubic) to ensure smoothness between data points.
4. **Lagrange interpolation:** A polynomial-based method that fits the given data points using Lagrange polynomials.

Interpolation is widely used in fields such as data science, engineering, and computer graphics to estimate missing data or smooth out noisy data

Interpolation has several important applications in computer science, particularly in areas that involve data processing, graphics, and numerical computations. Here are some key applications:

1. **Computer Graphics and Image Processing**
 - Texture Mapping
 - Rescaling and Resampling Images
 - Antialiasing
2. **Data Science and Machine Learning**
 - Missing Data Imputation
3. **Numerical Methods**
 - Solving Differential Equations
 - Numerical Integration and Differentiation
4. **Game Development and Physics Simulations**
 - Pathfinding Algorithms
 - Simulation
5. **Geographic Information Systems (GIS)**
 - Terrain Mapping and Modeling
 - Weather Forecasting
6. **Animation and Motion Capture**
 - Smooth Animation Transitions
 - Motion Blending
7. **Signal Processing**
 - Signal Reconstruction
8. **CAD (Computer-Aided Design) and CAM (Computer-Aided Manufacturing)**
 - Curve Fitting and Surface Modeling
9. **Cryptography**
 - Elliptic Curve Cryptography (ECC)
10. **Compression Algorithms**
 - Data Compression

Conclusion

Interpolation plays a crucial role in various domains within computer science, particularly where estimation of intermediate values is required, whether it's for generating smoother animations, analyzing data, or improving the quality of images and simulations.

Feature	Newton's Forward Interpolation	Newton's Backward Interpolation
Interpolation Focus	Used for interpolating values close to the start of the dataset, near x_0 .	Used for interpolating values close to the end of the dataset, near x_n .
Difference Table	Forward difference table is used.	Backward difference table is used.

Formula:

$$y_0 + P \Delta y_0 + \frac{P(P-1)}{2!} \Delta^2 y_0$$

$$P = \frac{x - x_0}{h}$$

$$y(x) = y_n + Pn \nabla y_n + \frac{P(P+1)}{2!} \Delta^2 y_n + \frac{P(P+1)(P+2)}{3!} \Delta^3 y_n$$

$$P = \frac{x - x_n}{h}$$

Application

computer graphics,
Signal processing
and machine
learning.

- ① handling Missing Data in Dataset
- ② Data Preprocessing for Time Series Forecasting
- ③ Feature Engineering in ML
- ④ Curve Fitting for Data visualization.
- ⑤ Model optimization and Hyperparameters Tuning.
- ⑥ Data Augmentation.

Time Series
Forecasting
Data preprocessing
eg cryptography
ML, computer
graphics.